جامعة أم القرى
UMM AL-QURA UNIVERSITY

Introduction to Database project:
University Buses

Section: 16
Group Members :

NAME	ID
Reham Khan	445013974
Lujain Zahid	445014050
Afrah Ayob	445013880
Maha Ahmed	445014009

Dr. Arwa Halawani

	Table of Contents:
1	ER MODEL (SCENARIO)
2	Section 1 : Conceptual and Logical Design 1.1 ER Diagram 1.2 Relational Schema
3	Section 2: Implementation 2.1 Creating the Database 2.2 Creating Tables 2.3 Inserting Data 2.4 SQL Queries

Responsibilities:	
ER MODEL (SCENARIO)	All team members worked together and shared responsibilities to complete the project.
ER Diagram	
Relational Schema	
SQL Tables	
Queries	
Report Writing	

• ER MODEL (SCENARIO):

University shuttle buses are a means of group transportation that transfer students from different residential areas to the university and back to their homes.
They are supervised by the university administration through drivers and supervisors, with each bus assigned to a specific route covering designated areas, ensuring safe and convenient transportation for student.

The records information about students, drivers, buses, areas, and the supervisors of the buses.

A **Student** is identified by their Student_ID, Name, Registration_Date, Department, and Phone number.
A **Driver** is identified by their Driver_ID, Name, Phone number, License Number, Salary, and Shift_Type.
A **Supervisor** is identified by their Supervisor_ID, Name, and Phone number.
A **Bus** is identified by its Bus_ID, Plate Number, Capacity, Status, and Model_Year.
An **Area** is identified by its Area_ID, Area_Name, and Area_Code (for example: SHJ for Al Hajj Street , WJG for Wadi Jaleel , UVC for University Campus)

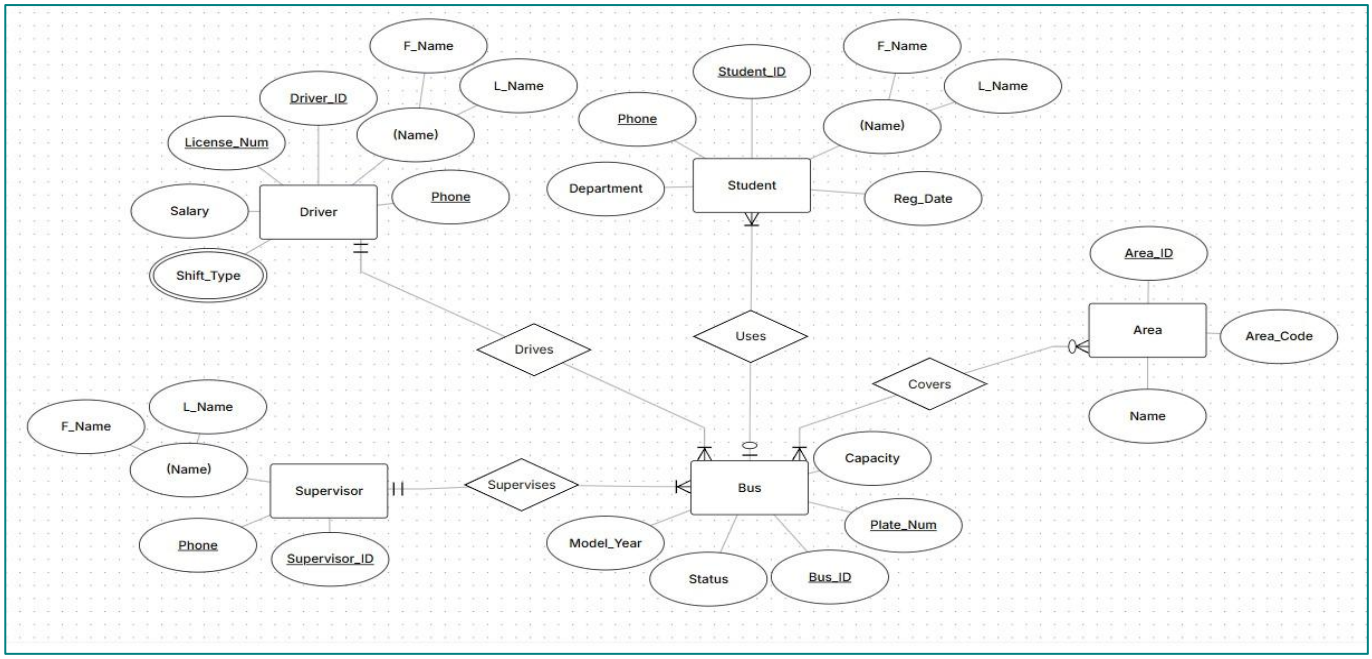
Relationships :
A Bus covers many Areas, and each Area can be served by many Buses (M:N relationship).
A Driver can drive multiple Buses, but each Bus is assigned to only one Driver (1:M relationship).
A Supervisor supervises multiple Buses, and each Bus is supervised by one Supervisor (1:M relationship).
Many students use one Bus, and one Bus carries multiple students (1:M relationship).

- **Database Structure Summary:**

Entity	Attributes	Relationships
Student	Student_ID, Name, Registration_Date, Department, Phone	Many Students use one Bus, and one Bus carries multiple Students (1:M)
Driver	Driver_ID, Name, Phone, License_Number, Salary, Shift_Type	A Driver can drive multiple Buses, but each Bus is assigned to one Driver (1:M)
Supervisor	Supervisor_ID, Name, Phone	A Supervisor supervises multiple Buses, and each Bus is supervised by one Supervisor (1:M)
Bus	Bus_ID, Plate_Number, Capacity, Status, Model_Year	A Bus covers many Areas, carries many Students, and is assigned to one Driver and one Supervisor
Area	Area_ID, Area_Name, Area_Code	A Bus covers many Areas, and each Area can be served by many Buses (M:N)

Section 1 : Conceptual and Logical Design

1.1 ER Diagram:



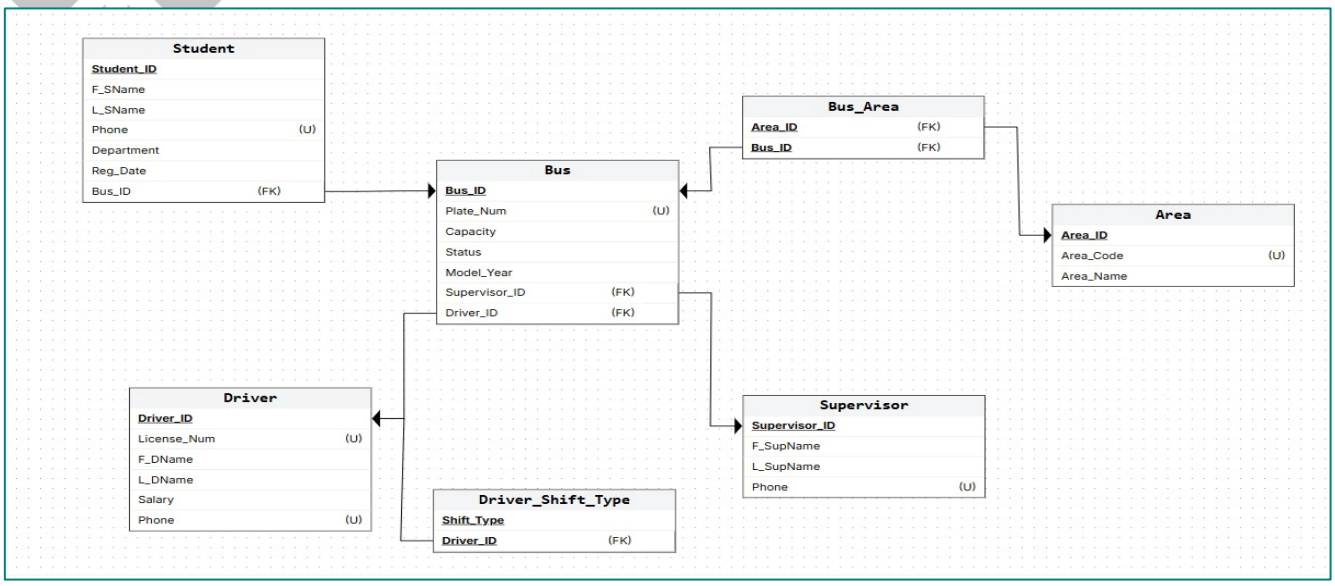
Driver – Bus (Drives):
Both are Mandatory: Every driver must drive a bus, and every bus must have a driver.

Supervisor – Bus (Supervises):
Both are Mandatory: Every supervisor must supervise a bus, and every bus must be supervised by a supervisor.

Area – Bus (Covers):
Area Mandatory: Every area must be covered by a bus.
Bus Optional: A bus does not have to cover every area

Student – Bus (Uses):
Student Optional: A student is not required to use a bus
Bus Mandatory: Every bus must have a student.

1.2 Relational Schema:



Relational Schema Design Description:

The **Student** entity uses Student_ID as a primary key. Since the relationship between Student and Bus is one-to-many, the Bus_ID was added to the Student table as a foreign key, indicating that multiple students can be assigned to the same bus.

The **Driver** entity uses Driver_ID as a primary key. Because a driver can drive multiple buses while each bus is assigned to one driver, the Driver_ID was added as a foreign key in the Bus table. The attribute Shift_Type was originally a multivalued attribute for the driver, so it was moved into a separate table called Driver_Shift_Type to maintain normalization. This table contains Driver_ID as a foreign key along with the shift type and both consider as a primary key in this table.

The **Bus** entity uses Bus_ID as a primary key and includes foreign keys to represent its relationships with other entities. The Supervisor_ID was added as a foreign key in the Bus table to represent the one-to-many relationship where one supervisor can supervise multiple buses.

The **Area** entity uses Area_ID as a primary key. Since the relationship between Bus and Area is many-to-many, an associative table named Bus_Area was created. This table includes Bus_ID and Area_ID as foreign keys to represent which buses serve which areas and both consider as a primary key in this table.

Relationship	Cardinality	Implementation in Schema
Student – Bus	One-to-Many	Bus_ID added as FK in Student
Driver – Bus	One-to-Many	Driver_ID added as FK in Bus
Supervisor – Bus	One-to-Many	Supervisor_ID added as FK in Bus
Driver – Shift_Type	Multivalued	Separate table (Driver_Shift_Type)
Bus – Area	Many-to-Many	Associative table (Bus_Area)

Section 2: Implementation

2.1 Creating the Database:

Explanation :
In this step, we created the main database and activated it using the USE command.
This ensures all subsequent SQL commands are executed inside the correct database.

```
-- Creating the University_Buses database and use i
create database University_Buses;
show databases;
use University_Buses;
```

The output:

Result Grid | Filter Rows:

Database
mysql
najed_delight
performance_schema
sys
university_buses

2.2 Creating Tables:

Explanation
We created all required tables based on the ERD.
Each table includes:
Primary Keys (PK)
Foreign Keys (FK)
UNIQUE constraints
CHECK constraints
ON UPDATE CASCADE
This maintains data integrity and ensures relationships work correctly.

- **Table: Area**

Explanation

Stores information about all residential areas covered by buses.

```sql
-- Table: Area
CREATE TABLE Area (
    Area_ID INT PRIMARY KEY,
    Area_Code VARCHAR(10) NOT NULL UNIQUE,
    Area_Name VARCHAR(100) not null
);
```

```sql
310 • describe Area;
```

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Area_ID	int	NO	PRI	NULL	
Area_Code	varchar(10)	NO	UNI	NULL	
Area_Name	varchar(100)	NO		NULL	

- **Table: Supervisor**

Explanation

Stores information about each bus supervisor including name and phone.

```sql
-- Table: Supervisor
CREATE TABLE Supervisor (
    Supervisor_ID INT PRIMARY KEY,
    F_SupName VARCHAR(50) not null,
    L_SupName VARCHAR(50) not null,
    Phone VARCHAR(20) UNIQUE
);
```

```sql
318 • describe Supervisor;
```

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Supervisor_ID	int	NO	PRI	NULL	
F_SupName	varchar(50)	NO		NULL	
L_SupName	varchar(50)	NO		NULL	
Phone	varchar(20)	YES	UNI	NULL	

- **Table: Driver**

Explanation

Stores driver details such as salary (must be > 0), license, and phone number.

```sql
-- Table: Driver
CREATE TABLE Driver (
    Driver_ID INT PRIMARY KEY,
    License_Num VARCHAR(30) UNIQUE NOT NULL
    F_DName VARCHAR(50) not null,
    L_DName VARCHAR(50) not null,
    Salary DECIMAL(10,2) CHECK (Salary > 0)
    Phone VARCHAR(20) UNIQUE
);
```

```sql
328 • describe Driver;
```

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Driver_ID	int	NO	PRI	NULL	
License_Num	varchar(30)	NO	UNI	NULL	
F_DName	varchar(50)	NO		NULL	
L_DName	varchar(50)	NO		NULL	
Salary	decimal(10,2)	YES		NULL	
Phone	varchar(20)	YES	UNI	NULL	

- **Table: Bus**

Explanation

Stores all bus information and links each bus to a driver and supervisor.

Driver_ID → FK from Driver table

Supervisor_ID → FK from Supervisor table

Both have ON UPDATE CASCADE so changes update automatically.

```sql
-- Table: Bus
CREATE TABLE Bus (
    Bus_ID INT PRIMARY KEY,
    Plate_Num VARCHAR(20) UNIQUE,
    Capacity INT,
    Status VARCHAR(50),
    Model_Year INT,
    Driver_ID INT,
    Supervisor_ID INT,
    FOREIGN KEY (Driver_ID) REFERENCES Driver(Driver_ID) ON UPDATE CASCADE,
    FOREIGN KEY (Supervisor_ID) REFERENCES Supervisor(Supervisor_ID) ON UPDATE CASCADE
);
```

```sql
342 • describe Bus;
```

Result Grid | Filter Rows: | Export: | W

Field	Type	Null	Key	Default	Extra
Bus_ID	int	NO	PRI	NULL	
Plate_Num	varchar(20)	YES	UNI	NULL	
Capacity	int	YES		NULL	
Status	varchar(50)	YES		NULL	
Model_Year	int	YES		NULL	
Driver_ID	int	YES	MUL	NULL	
Supervisor_ID	int	YES	MUL	NULL	

- **Table: Student**

Explanation

Stores student information and links each student to a bus.

```
-- Table: Student
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    F_SName VARCHAR(50) not null,
    L_SName VARCHAR(50) not null,
    Phone VARCHAR(20) UNIQUE,
    Department VARCHAR(50),
    Reg_Date DATE,
    Bus_ID INT,
    FOREIGN KEY (Bus_ID) REFERENCES Bus(Bus_ID) ON UPDATE CASCADI
);
```

```
355 •  describe Student;
```

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
Student_ID	int	NO	PRI	NULL	
F_SName	varchar(50)	NO		NULL	
L_SName	varchar(50)	NO		NULL	
Phone	varchar(20)	YES	UNI	NULL	
Department	varchar(50)	YES		NULL	
Reg_Date	date	YES		NULL	
Bus_ID	int	YES	MUL	NULL	

- **Table: Bus_Area (M:N Relationship)**

Explanation

A bus can cover many areas, and an area can be covered by multiple buses.
We used a composite primary key.

```
-- Table: Bus_Area
CREATE TABLE Bus_Area (
    Bus_ID INT,
    Area_ID INT,
    PRIMARY KEY (Bus_ID, Area_ID),
    FOREIGN KEY (Bus_ID) REFERENCES Bus(Bus_ID)  ON UPDATE CASCADE,
    FOREIGN KEY (Area_ID) REFERENCES Area(Area_ID)  ON UPDATE CASCADI
);
```

```
365 •  describe Bus_Area;
```

Result Grid | Filter Rows: | Export

Field	Type	Null	Key	Default	Extra
Bus_ID	int	NO	PRI	NULL	
Area_ID	int	NO	PRI	NULL	

- **Table: Driver_Shift_Type**

Explanation

Represents the multi-valued attribute “shift type” for each driver.

```
-- Table: Driver_Shift_Type
CREATE TABLE Driver_Shift_Type (
    Shift_Type VARCHAR(30),
    Driver_ID INT,
    PRIMARY KEY (Shift_Type, Driver_ID),
    FOREIGN KEY (Driver_ID) REFERENCES Driver(Driver_ID)  ON UPDATE CASCADE
);
```

```
375 •  describe Driver_Shift_Type;
```

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
Shift_Type	varchar(30)	NO	PRI	NULL	
Driver_ID	int	NO	PRI	NULL	

Tables in university buses:

Tables_in_university_buses
area
bus
bus_area
driver
driver_shift_type
student
student_bus_supervisor_student1
supervisor

2.3 Inserting Data

Explanation
We inserted **10 records** per table.

- **Insert Into DRIVER Table**

Explanation:
This command inserts a new driver into the **DRIVER** table.
It includes the driver's ID, name, phone number, license number, salary, and shift type.
Each value must satisfy the table rules, such as unique License_Number, valid phone number format.

```
- Drivers
NSERT INTO Driver (Driver_ID, License_Num, F_DName, L_DName, Salary, Phone) VALUES
1,'1023456789','Ahmed','Salem',4500,'055100001'),
2,'1022233344','Mazen','Ali',4200,'055100002'),
3,'1033344455','Fahad','Khaled',5000,'055100003'),
4,'1044455566','Saad','Hussein',3900,'055100004'),
5,'1055566677','Khaled','Majed',4700,'055100005'),
6,'1066677788','Amer','Omar',4100,'055100006'),
7,'1077788899','Sultan','Rashid',5500,'055100007'),
8,'1088899911','Adeab','Faisal',4300,'055100008'),
9,'1076584936','Talal','Amin',4600,'055100009'),
10,'109864556','Salem','Saad',4400,'055100010');
```

The output:

```
302 •  select * from driver;
303
```

Result Grid | Filter Rows: | Edit:

Driver_ID	License_Num	F_DName	L_DName	Salary	Phone
1	1023456789	Ahmed	Salem	4500.00	055100001
2	1022233344	Mazen	Ali	4200.00	055100002
3	1033344455	Fahad	Khaled	5000.00	055100003
4	1044455566	Saad	Hussein	3900.00	055100004
5	1055566677	Khaled	Majed	4700.00	055100005
6	1066677788	Amer	Omar	4100.00	055100006
7	1077788899	Sultan	Rashid	5500.00	055100007
8	1088899911	Adeab	Faisal	4300.00	055100008
9	1076584936	Talal	Amin	4600.00	055100009
10	109864556	Salem	Saad	4400.00	055100010

- **Insert Into DRIVER_SHIFT_TYPE Table**

Explanation:

This INSERT statement adds multiple records into the **DRIVER_SHIFT_TYPE** table, which stores the shift assignments for each driver.

The table links a driver to a specific shift type (such as *Morning* or *Evening*).

```
- Driver shift types
INSERT INTO Driver_Shift_Type (Shift_Type, Driver_ID) VALUE
('Morning',1),
('Evening',1),
('Morning',2 ),
('Evening',3),
('Morning',4),
('Morning',5),
('Evening',6),
('Morning',7),
('Morning',8),
('Evening',9);
```

The output:

```
316 •  select * from driver_shift_type;
317
```

Result Grid | Filter Rows:

Shift_Type	Driver_ID
Evening	1
Evening	3
Evening	6
Evening	9
Morning	1
Morning	2
Morning	4
Morning	5
Morning	7
Morning	8

- **Insert Into SUPERVISOR Table**

Explanation:

This INSERT adds a new supervisor into the **SUPERVISOR** table.
The record includes a supervisor ID, full name, and phone number.

```sql
-- Supervisors
INSERT INTO Supervisor (Supervisor_ID, F_SupName, L_SupName, Phone)  VALUES
(1,'Lujain','Hassan','055200001'),
(2,'Reham','Yousef','055200002'),
(3,'Reema','Ali','055200003'),
(4,'Afrah','Fahad','055200004'),
(5,'Fatimah','Saleh','055200005'),
(6,'Maha','Sami','055200006'),
(7,'Nawal','Qasim','055200007'),
(8,'Areej','Mahmoud','055200008'),
(9,'Laila','Hadi','055200009'),
(10,'Rania','Imad','055200010');
```

The output:

```sql
select * from supervisor;
```

Result Grid | Filter Rows: | Edit:

Supervisor_ID	F_SupName	L_SupName	Phone
1	Lujain	Hassan	055200001
2	Reham	Yousef	055200002
3	Reema	Ali	055200003
4	Afrah	Fahad	055200004
5	Fatimah	Saleh	055200005
6	Maha	Sami	055200006
7	Nawal	Qasim	055200007
8	Areej	Mahmoud	055200008
9	Laila	Hadi	055200009
10	Rania	Imad	055200010
NULL	NULL	NULL	NULL

- **Insert Into BUS Table**

Explanation:

This statement inserts a new bus entry into the **BUS** table.
All bus details are provided including Bus_ID, plate number, capacity, status, model year, and references to the assigned driver and supervisor.
The Driver_ID and Supervisor_ID must already exist in their respective tables to maintain referential integrity.

```
- buses
NSERT INTO Bus (Bus_ID, Plate_Num, Capacity, Status, Model_Year, Driver_ID, Supervisor_ID) VALUES
1, 'SAU100', 45, 'Active', 2018, 1, 1),
2, 'MAD101', 50, 'Active', 2019, 2, 2),
3, 'MNH102', 40, 'Inactive', 2017, 3, 3),
4, 'RER103', 48, 'Active', 2020, 4, 4),
5, 'FOF104', 55, 'Active', 2021, 5, 5),
6, 'LOL105', 42, 'Inactive', 2016, 6, 6),
7, 'SAR106', 47, 'Active', 2022, 7,7),
8, 'ILK107', 50, 'Active', 2020, 8,8),
9, 'ZDT108', 44, 'Inactive', 2015, 9,9),
10, 'AFA109', 60, 'Active', 2023, 10,10);
```

The output:

```
358 •   select * from bus;
359
```

Result Grid | Filter Rows: | Edit: | Export/Impor

Bus_ID	Plate_Num	Capacity	Status	Model_Year	Driver_ID	Supervisor_ID
1	SAU100	45	Active	2018	1	1
2	MAD101	50	Active	2019	2	2
3	MNH102	40	Inactive	2017	3	3
4	RER103	48	Active	2020	4	4
5	FOF104	55	Active	2021	5	5
6	LOL105	42	Inactive	2016	6	6
7	SAR106	47	Active	2022	7	7
8	ILK107	50	Active	2020	8	8
9	ZDT108	44	Inactive	2015	9	9
10	AFA109	60	Active	2023	10	10

- **Insert Into AREA Table**

Explanation:

This INSERT command adds a new area into the **AREA** table.
It stores the area's unique ID, area name, and its official three-letter code.

```sql
-- Areas
INSERT INTO Area (Area_ID, Area_Code, Area_Name) VALUE
(1,'SHJ','Al Hajj Street'),
(2, 'RSH', 'Al Rashidiya'),
(3, 'HSY', 'Al Husayniya'),
(4, 'KHD', 'Al Khalidiyah'),
(5, 'SHR', 'Al Sharaie'),
(6, 'HMR', 'Al Hamra'),
(7, 'SHQ', 'Al Shawqiya'),
(8, 'BQR', 'Batha Quraysh'),
(9,'WJG','Wadi Jaleel'),
(10,'AUL','Al Auali');
```

The output:

```sql
select * from area;
```

Result Grid | Filter Rows:

Area_ID	Area_Code	Area_Name
1	SHJ	Al Hajj Street
2	RSH	Al Rashidiya
3	HSY	Al Husayniya
4	KHD	Al Khalidiyah
5	SHR	Al Sharaie
6	HMR	Al Hamra
7	SHQ	Al Shawqiya
8	BQR	Batha Quraysh
9	WJG	Wadi Jaleel
10	AUL	Al Auali

- **Insert Into BUS_AREA (M:N Relationship Table)**

Explanation:

This statement inserts a new record into the **BUS_AREA** linking table, which represents the many-to-many relationship between buses and areas.
Each record associates one bus with one specific area.
Both Bus_ID and Area_ID must exist in their parent tables, and the pair must not be duplicated due to the composite primary key.

```
-- bus_area relations
INSERT INTO Bus_Area (Bus_ID, Area_ID)  VALUES
(1,1),
(1,2),
(2,3),
(2,4),
(3,5),
(3,1),
(4,6),
(5,7),
(6,8),
(7,9),
(8,10),
(9,3),
(10,2),
(10,1),
(7,4);
```

The outpute:

```
377 •   select * from bus_area;
378
```

Result Grid | Filter Rows:

Bus_ID	Area_ID
1	1
1	2
2	3
2	4
3	1
3	5
4	6
5	7
6	8
7	4

bus_area 9

- **Insert Into STUDENT Table**

Explanation:

This INSERT statement adds a new student record into the **STUDENT** table.
It provides all required attributes: the student's unique ID, full name, registration date, department, phone number, and the bus they are assigned to.
The values inserted must follow the table constraints such as NOT NULL, correct data types, and an existing Bus_ID (foreign key).

```sql
-- students
INSERT INTO Student(Student_ID, F_SName, L_SName, Phone, Department, Reg_Date, Bus_ID) VALUES
(1001,'Areej','Ahmad','056300001','IT','2024-09-01',1),
(1002,'Maha','Omar','056300002','CS','2024-09-01',1),
(1003,'Rama','Saeed','056300003','AI','2024-09-02',2),
(1004,'Aya','Saleh','056300004','IT','2024-09-01',3),
(1005,'Nour','Fahad','056300005','AI','2024-09-04',3),
(1006,'Ruba','Majed','056300006','CS','2024-09-01',4),
(1007,'Kholod','Nasser','056300007','IT','2024-09-01',5),
(1008,'Sama','Ali','056300008','AI','2024-09-05',6),
(1009,'Lujain','Yousef','056300009','AI','2024-09-01',7),
(1010,'Lama','Hasan','056300010','CS','2024-09-01',10);
```

The output:

```sql
select * from student;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

Student_ID	F_SName	L_SName	Phone	Department	Reg_Date	Bus_ID
1001	Areej	Ahmad	056300001	IT	2024-09-01	1
1002	Maha	Omar	056300002	CS	2024-09-01	1
1003	Rama	Saeed	056300003	AI	2024-09-02	2
1004	Aya	Saleh	056300004	IT	2024-09-01	3
1005	Nour	Fahad	056300005	AI	2024-09-04	3
1006	Ruba	Majed	056300006	CS	2024-09-01	4
1007	Kholod	Nasser	056300007	IT	2024-09-01	5
1008	Sama	Ali	056300008	AI	2024-09-05	6
1009	Lujain	Yousef	056300009	AI	2024-09-01	7
1010	Lama	Hasan	056300010	CS	2024-09-01	10

2.4 SQL Queries:

Maha – SQL Queries Explanation

- **Basic SELECT Queries**

Query: Retrieve first name, last name, and phone of students in the AI department.

Explanation:

This query selects all students who belong to the AI department.
It returns their first name, last name, and phone number.
The WHERE Department = 'AI' condition filters the records so only AI students appear in the result.

```
SELECT F_SName, L_SName, Phone
FROM Student
WHERE Department = 'AI';
```

The output:

Result Grid | Filter Rows:

F_SName	L_SName	Phone
Rama	Saeed	056300003
Nour	Fahad	056300005
Sama	Ali	056300008
Lujain	Yousef	056300009

Query: Select all active buses with their ID, plate number, and status

Explanation:

This query retrieves all buses whose status is “Active.”
It displays the bus ID, plate number, and status.
This is useful for identifying which buses are currently operating.

```
SELECT Bus_ID, Plate_Num, Status
FROM Bus
WHERE Status = 'Active';
```

The output:

Result Grid | Filter Rows:

Bus_ID	Plate_Num	Status
1	SAU100	Active
2	MAD101	Active
4	RER103	Active
5	FOF104	Active
7	SAR106	Active
8	ILK107	Active
10	AFA109	Active
NULL	NULL	NULL

- **Aggregate Function Queries**

Query: Count how many students are assigned to each bus

Explanation:

This query groups students by their assigned bus and counts how many students ride each bus.

Using COUNT(*) with GROUP BY Bus_ID provides useful statistics for bus capacity planning.

```
SELECT Bus_ID, COUNT(*) AS NumberOfStudents
FROM Student
GROUP BY Bus_ID;
```

The output:

Result Grid | Filter Rows:

Bus_ID	NumberOfStudents
1	2
2	1
3	2
4	1
5	1
6	1
7	1
10	1

Query: Calculate the average salary of all drivers

Explanation:

This query calculates the average salary in the Driver table.

It uses AVG(Salary) to show overall salary trends and helps in salary analysis.

```
SELECT AVG(Salary) AS AvgSalary
FROM Driver;
```

The output:

AvgSalary
4520.000000

- **JOIN Queries**

Query: Show all drivers and the buses they are assigned to (LEFT JOIN)

Explanation:

This query displays every driver, whether assigned to a bus or not.
A LEFT JOIN is used so drivers without assigned buses still appear with NULL values.
This is useful for detecting idle or unassigned drivers.

```
SELECT d.F_DName, d.L_DName, b.Bus_ID
FROM Driver d
LEFT JOIN Bus b ON d.Driver_ID = b.Driver_ID;
```

The output:

F_DName	L_DName	Bus_ID
Ahmed	Salem	1
Mazen	Ali	2
Fahad	Khaled	3
Saad	Hussein	4
Khaled	Majed	5
Amer	Omar	6
Sultan	Rashid	7
Adeab	Faisal	8
Talal	Amin	9
Salem	Saad	10

Query: Show students with their bus plate number and status (INNER JOIN)

Explanation:

This query returns all students along with details of the bus they use (plate number and status).
The INNER JOIN ensures only students who are assigned to a valid bus appear.

```
SELECT s.F_SName, s.L_SName, b.Plate_Num, b.Status
FROM Student s
INNER JOIN Bus b ON s.Bus_ID = b.Bus_ID;
```

The output:

F_SName	L_SName	Plate_Num	Status
Areej	Ahmad	SAU100	Active
Maha	Omar	SAU100	Active
Rama	Saeed	MAD101	Active
Aya	Saleh	MNH102	Inactive
Nour	Fahad	MNH102	Inactive
Ruba	Majed	RER103	Active
Kholod	Nasser	FOF104	Active
Sama	Ali	LOL105	Inactive
Lujain	Yousef	SAR106	Active
Lama	Hasan	AFA109	Active

- **Subquery Queries**

Query: Retrieve students assigned to buses with above-average capacity

Explanation:

This subquery checks which buses have a capacity greater than the average bus capacity, then retrieves all students riding those buses.

It helps analyze student distribution on large buses.

```
SELECT *
FROM Student
WHERE Bus_ID IN (
    SELECT Bus_ID
    FROM Bus
    WHERE Capacity > (SELECT AVG(Capacity) FROM Bus)
);
```

The output:

Result Grid | Filter Rows: | Edit: | Export/Import:

Student_ID	F_SName	L_SName	Phone	Department	Reg_Date	Bus_ID
1003	Rama	Saeed	056300003	AI	2024-09-02	2
1007	Kholod	Nasser	056300007	IT	2024-09-01	5
1010	Lama	Hasan	056300010	CS	2024-09-01	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query: Retrieve supervisors who oversee more than one bus

Explanation:

This subquery identifies supervisors responsible for multiple buses by grouping buses by supervisor and counting them.

Only supervisors with more than one bus are returned.

```
SELECT Supervisor_ID, F_SupName, L_SupName
FROM Supervisor
WHERE Supervisor_ID IN (
    SELECT Supervisor_ID
    FROM Bus
    GROUP BY Supervisor_ID
    HAVING COUNT(*) > 1
);
```

The output:

Supervisor_ID	F_SupName	L_SupName
NULL	NULL	NULL

- **VIEW Query**

Query: View showing each student with their bus and supervisor names

Explanation:

This view combines three tables: Student, Bus, and Supervisor.

It displays each student along with their bus plate number and the supervisor who manages that bus.

Views like this simplify reporting by showing commonly needed information in one place.

```
CREATE OR REPLACE VIEW Student_Bus_Supervisor_Student1 AS
SELECT
    s.Student_ID,
    s.F_SName,
    s.L_SName,
    b.Plate_Num,
    b.Status,
    sp.F_SupName AS Supervisor_FirstName,
    sp.L_SupName AS Supervisor_LastName
FROM Student s
INNER JOIN Bus b ON s.Bus_ID = b.Bus_ID
INNER JOIN Supervisor sp ON b.Supervisor_ID = sp.Supervisor_ID;
```

The output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Student_ID	F_SName	L_SName	Plate_Num	Status	Supervisor_FirstName	Supervisor_LastName
1001	Areej	Ahmad	SAU100	Active	Lujain	Hassan
1002	Maha	Omar	SAU100	Active	Lujain	Hassan
1003	Rama	Saeed	MAD101	Active	Reham	Yousef
1004	Aya	Saleh	MNH102	Inactive	Reema	Ali
1005	Nour	Fahad	MNH102	Inactive	Reema	Ali
1006	Ruba	Majed	RER103	Active	Afrah	Fahad
1007	Kholod	Nasser	FOF104	Active	Fatimah	Saleh
1008	Sama	Ali	LOL105	Inactive	Maha	Sami
1009	Lujain	Yousef	SAR106	Active	Nawal	Qasim
1010	Lama	Hasan	AFA109	Active	Rania	Imad

Afrah – SQL Queries Explanation

- **Basic SELECT Queries**

Query: Retrieve students in the IT department.

Explanation:

This query selects all students in the IT department, retrieving their first name, last name, and department.

The filter condition ensures the output includes only IT students.

```
SELECT F_SName, L_SName, Department
FROM Student
WHERE Department = 'IT';
```

The output:

Result Grid | Filter Rows:

F_SName	L_SName	Department
Areej	Ahmad	IT
Aya	Saleh	IT
Kholod	Nasser	IT

Query: Select buses with status 'active' and capacity > 45

Explanation:

This query returns active buses that have a large capacity.

The combined conditions help identify suitable buses for high-demand routes.

```
SELECT Bus_ID, Plate_Num, Capacity
FROM Bus
WHERE Status = 'active' AND Capacity > 45;
```

The output:

Bus_ID	Plate_Num	Capacity
2	MAD101	50
4	RER103	48
5	FOF104	55
7	SAR106	47
8	ILK107	50
10	AFA109	60
NULL	NULL	NULL

- **Aggregate Function Queries**

Query: Find the minimum driver salary

Explanation:

This query identifies the lowest salary in the Driver table using MIN(Salary).
It is useful for payroll analysis and salary structure evaluation.

```
SELECT MIN(Salary) AS MinDriverSalary
FROM Driver;
```

The output:

	MinDriverSalary
▶	3900.00

Query: Find maximum bus capacity

Explanation:

This query returns the largest capacity value among all buses.
It helps understand transportation capacity limits.

```
SELECT MAX(Capacity) AS MaxBusCapacity
FROM Bus;
```

The output:

	MaxBusCapacity
▶	60

- **JOIN Queries**

Query: RIGHT JOIN between Supervisor and Bus

Explanation:

This query lists all buses and adds supervisor information if available.

The RIGHT JOIN ensures all buses appear, even if they don't have a supervisor assigned.

```
SELECT b.Bus_ID, b.Plate_Num, sp.F_SupName, sp.Phone
FROM Supervisor sp
RIGHT JOIN Bus b ON sp.Supervisor_ID = b.Supervisor_ID;
```

The output:

Bus_ID	Plate_Num	F_SupName	Phone
1	SAU100	Lujain	055200001
2	MAD101	Reham	055200002
3	MNH102	Reema	055200003
4	RER103	Afrah	055200004
5	FOF104	Fatimah	055200005
6	LOL105	Maha	055200006
7	SAR106	Nawal	055200007
8	ILK107	Areej	055200008
9	ZDT108	Laila	055200009
10	AFA109	Rania	055200010

Query: Students with their driver names (LEFT JOIN)

Explanation:

This retrieves each student along with the name of the driver responsible for their bus.

A LEFT JOIN ensures all students appear even if a bus has no driver assigned.

```
SELECT s.F_SName, s.L_SName, d.F_DName, d.L_DName
FROM Student s
LEFT JOIN Bus b ON s.Bus_ID = b.Bus_ID
LEFT JOIN Driver d ON b.Driver_ID = d.Driver_ID;
```

The output:

F_SName	L_SName	F_DName	L_DName
Areej	Ahmad	Ahmed	Salem
Maha	Omar	Ahmed	Salem
Rama	Saeed	Mazen	Ali
Aya	Saleh	Fahad	Khaled
Nour	Fahad	Fahad	Khaled
Ruba	Majed	Saad	Hussein
Kholod	Nasser	Khaled	Majed
Sama	Ali	Amer	Omar
Lujain	Yousef	Sultan	Rashid
Lama	Hasan	Salem	Saad

- **Subquery Queries**

Query: Students assigned to buses with maximum capacity

Explanation:

This query finds the bus (or buses) with the maximum capacity, then retrieves all students assigned to those buses.

```
SELECT *
FROM Student
WHERE Bus_ID IN (
    SELECT Bus_ID
    FROM Bus
    WHERE Capacity = (SELECT MAX(Capacity) FROM Bus)
);
```

The output:

Student_ID	F_SName	L_SName	Phone	Department	Reg_Date	Bus_ID
1010	Lama	Hasan	056300010	CS	2024-09-01	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query: Supervisors who manage buses with minimum capacity

Explanation:

This subquery identifies supervisors responsible for buses with the lowest capacity. It cross-checks the Supervisor and Bus tables using matching IDs.

```
SELECT Supervisor_ID, F_SupName, L_SupName
FROM Supervisor
WHERE Supervisor_ID IN (
    SELECT Supervisor_ID
    FROM Bus
    WHERE Capacity = (SELECT MIN(Capacity) FROM Bus)
);
```

The output:

Supervisor_ID	F_SupName	L_SupName
3	Reema	Ali
NULL	NULL	NULL

- **VIEW Query**

Query: Bus view with supervisor and driver names

Explanation:

This view displays full details for each bus, including supervisor and driver names.
It is useful as a summarized operational report combining three tables.

```
CREATE OR REPLACE VIEW Bus_Student4_View AS
SELECT
    b.Bus_ID,
    b.Plate_Num,
    b.Status,
    sp.F_SupName AS Supervisor_FirstName,
    sp.L_SupName AS Supervisor_LastName,
    d.F_DName AS Driver_FirstName,
    d.L_DName AS Driver_LastName
FROM Bus b
LEFT JOIN Supervisor sp ON b.Supervisor_ID = sp.Supervisor_ID
LEFT JOIN Driver d ON b.Driver_ID = d.Driver_ID;
```

The output:

Bus_ID	Plate_Num	Status	Supervisor_FirstName	Supervisor_LastName	Driver_FirstName	Driver_LastName
1	SAU100	Active	Lujain	Hassan	Ahmed	Salem
2	MAD101	Active	Reham	Yousef	Mazen	Ali
3	MNH102	Inactive	Reema	Ali	Fahad	Khaled
4	RER103	Active	Afrah	Fahad	Saad	Hussein
5	FOF104	Active	Fatimah	Saleh	Khaled	Majed
6	LOL105	Inactive	Maha	Sami	Amer	Omar
7	SAR106	Active	Nawal	Qasim	Sultan	Rashid
8	ILK107	Active	Areej	Mahmoud	Adeab	Faisal
9	ZDT108	Inactive	Laila	Hadi	Talal	Amin
10	AFA109	Active	Rania	Imad	Salem	Saad

Reham – SQL Queries Explanation

- **Basic SELECT Queries**

Query: Students in the CS department

Explanation:

Returns all CS students with their first name, last name, and registration date.
This helps analyze enrollment in the CS department.

```
SELECT F_SName, L_SName, Reg_Date
FROM Student
WHERE Department = 'CS';
```

The output:

	F_SName	L_SName	Reg_Date
▶	Maha	Omar	2024-09-01
	Ruba	Majed	2024-09-01
	Lama	Hasan	2024-09-01

Query: Inactive buses with ID, plate number, and capacity

Explanation:

Shows all buses that are currently inactive along with capacity details.
Useful for maintenance planning.

```
SELECT Bus_ID, Plate_Num, Capacity
FROM Bus
WHERE Status = 'Inactive';
```

The output:

	Bus_ID	Plate_Num	Capacity
▶	3	MNH102	40
	6	LOL105	42
	9	ZDT108	44
*	NULL	NULL	NULL

- **Aggregate Function Queries**

Query: Count how many buses each driver is responsible for

Explanation:

This groups buses by driver and counts them.

It helps determine workload distribution among drivers.

```
SELECT Driver_ID, COUNT(Bus_ID) AS AssignedBuses
FROM Bus
GROUP BY Driver_ID;
```

The output:

Driver_ID	AssignedBuses
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1

Query: Total salary of all drivers

Explanation:

This calculates the salary budget for all drivers combined.

```
SELECT SUM(Salary) AS TotalDriverSalary
FROM Driver;
```

The output:

TotalDriverSalary
45200.00

- **JOIN Queries**

Query: Buses with their assigned driver (LEFT JOIN)

Explanation:

Shows every bus and driver names if the bus has a driver.

A left join ensures all buses appear, including those without drivers.

```
SELECT b.Bus_ID, b.Plate_Num, d.F_DName, d.L_DName
FROM Bus b
LEFT JOIN Driver d ON b.Driver_ID = d.Driver_ID;
```

The output:

Bus_ID	Plate_Num	F_DName	L_DName
1	SAU100	Ahmed	Salem
2	MAD101	Mazen	Ali
3	MNH102	Fahad	Khaled
4	RER103	Saad	Hussein
5	FOF104	Khaled	Majed
6	LOL105	Amer	Omar
7	SAR106	Sultan	Rashid
8	ILK107	Adeab	Faisal
9	ZDT108	Talal	Amin
10	AFA109	Salem	Saad

Query: Students with bus plate number and supervisor last name

Explanation:

Combines Student, Bus, and Supervisor data to show full routing and supervision details.

```
SELECT s.F_SName, s.L_SName, b.Plate_Num, sp.L_SupName
FROM Student s
INNER JOIN Bus b ON s.Bus_ID = b.Bus_ID
INNER JOIN Supervisor sp ON b.Supervisor_ID = sp.Supervisor_ID;
```

The output:

F_SName	L_SName	Plate_Num	L_SupName
Areej	Ahmad	SAU100	Hassan
Maha	Omar	SAU100	Hassan
Rama	Saeed	MAD101	Yousef
Aya	Saleh	MNH102	Ali
Nour	Fahad	MNH102	Ali
Ruba	Majed	RER103	Fahad
Kholod	Nasser	FOF104	Saleh
Sama	Ali	LOL105	Sami
Lujain	Yousef	SAR106	Qasim
Lama	Hasan	AFA109	Imad

- **Subquery Queries**

Query: Students assigned to buses with capacity < 50

Explanation:

Retrieves all students riding smaller buses by checking bus capacities through a subquery.

```
SELECT *
FROM Student
WHERE Bus_ID IN (
    SELECT Bus_ID
    FROM Bus
    WHERE Capacity < 50
);
```

The output:

Student_ID	F_SName	L_SName	Phone	Department	Reg_Date	Bus_ID
1001	Areej	Ahmad	056300001	IT	2024-09-01	1
1002	Maha	Omar	056300002	CS	2024-09-01	1
1004	Aya	Saleh	056300004	IT	2024-09-01	3
1005	Nour	Fahad	056300005	AI	2024-09-04	3
1006	Ruba	Majed	056300006	CS	2024-09-01	4
1008	Sama	Ali	056300008	AI	2024-09-05	6
1009	Lujain	Yousef	056300009	AI	2024-09-01	7
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query: Supervisors who oversee exactly one bus

Explanation:

Identifies supervisors who manage only one bus by grouping and filtering with HAVING COUNT = 1.

```
SELECT Supervisor_ID, F_SupName, L_SupName
FROM Supervisor
WHERE Supervisor_ID IN (
    SELECT Supervisor_ID
    FROM Bus
    GROUP BY Supervisor_ID
    HAVING COUNT(*) = 1
);
```

The output:

Supervisor_ID	F_SupName	L_SupName
1	Lujain	Hassan
2	Reham	Yousef
3	Reema	Ali
4	Afrah	Fahad
5	Fatimah	Saleh
6	Maha	Sami
7	Nawal	Qasim
8	Areej	Mahmoud
9	Laila	Hadi
10	Rania	Imad
NULL	NULL	NULL

- **VIEW Query**

Query: View showing each bus with driver and supervisor names

Explanation:

Creates a complete combined view summarizing bus info with driver and supervisor details.

Useful for administrative reporting.

```
CREATE OR REPLACE VIEW Bus_Driver_Supervisor_Student2 AS
SELECT
    b.Bus_ID,
    b.Plate_Num,
    b.Capacity,
    d.F_DName AS Driver_FirstName,
    d.L_DName AS Driver_LastName,
    sp.F_SupName AS Supervisor_FirstName,
    sp.L_SupName AS Supervisor_LastName
FROM Bus b
LEFT JOIN Driver d ON b.Driver_ID = d.Driver_ID
INNER JOIN Supervisor sp ON b.Supervisor_ID = sp.Supervisor_ID;
```

The output:

Bus_ID	Plate_Num	Capacity	Driver_FirstName	Driver_LastName	Supervisor_FirstName	Supervisor_LastName
1	SAU100	45	Ahmed	Salem	Lujain	Hassan
2	MAD101	50	Mazen	Ali	Reham	Yousef
3	MNH102	40	Fahad	Khaled	Reema	Ali
4	RER103	48	Saad	Hussein	Afrah	Fahad
5	FOF104	55	Khaled	Majed	Fatimah	Saleh
6	LOL105	42	Amer	Omar	Maha	Sami
7	SAR106	47	Sultan	Rashid	Nawal	Qasim
8	ILK107	50	Adeab	Faisal	Areej	Mahmoud
9	ZDT108	44	Talal	Amin	Laila	Hadi
10	AFA109	60	Salem	Saad	Rania	Imad

Lujain – SQL Queries Explanation

- **Basic SELECT Queries**

Query: Retrieve all shift types for driver 1

Explanation:

This query returns all the shift types assigned to driver ID 1.
Useful for reviewing schedule assignments.

```
SELECT Shift_Type, Driver_ID
FROM Driver_Shift_Type
WHERE Driver_ID = 1;
```

The output:

Shift_Type	Driver_ID
Morning	1
Evening	1
NULL	NULL

Query: Retrieve areas with codes starting with 'SH'

Explanation:

This query filters areas using LIKE 'SH%' to find all areas whose code begins with "SH."

```
SELECT Area_ID, Area_Code, Area_Name
FROM Area
WHERE Area_Code LIKE 'SH%';
```

The output:

Result Grid | Filter Rows:

Area_ID	Area_Code	Area_Name
1	SHJ	Al Hajj Street
7	SHQ	Al Shawqiya
5	SHR	Al Sharaie
NULL	NULL	NULL

- **Aggregate Function Queries**

Query: Count how many buses cover each area

Explanation:

Counts the number of buses serving each area by grouping records in Bus_Area.
Useful for analyzing route coverage.

```
SELECT Area_ID, COUNT(Bus_ID) AS NumberOfBuses
FROM Bus_Area
GROUP BY Area_ID;
```

The output:

Area_ID	NumberOfBuses
1	3
2	2
3	2
4	2
5	1
6	1
7	1
8	1
9	1
10	1

Query: Maximum number of shifts any driver has

Explanation:

A subquery counts shifts per driver, and the main query retrieves the maximum value.
This shows which driver has the busiest schedule.

```
SELECT MAX(NumShifts) AS MaxShifts
FROM (
    SELECT Driver_ID, COUNT(Shift_Type) AS NumShifts
    FROM Driver_Shift_Type
    GROUP BY Driver_ID
) AS DriverShifts;
```

The output:

MaxShifts
2

- **JOIN Queries**

Query: Buses with their areas (INNER JOIN)

Explanation:

This joins Bus, Bus_Area, and Area tables to list all buses and their corresponding areas.

```
SELECT b.Bus_ID, b.Plate_Num, a.Area_Name
FROM Bus b
INNER JOIN Bus_Area ba ON b.Bus_ID = ba.Bus_ID
INNER JOIN Area a ON ba.Area_ID = a.Area_ID;
```

The output:

Bus_ID	Plate_Num	Area_Name
1	SAU100	Al Hajj Street
1	SAU100	Al Rashidiya
2	MAD101	Al Husayniya
2	MAD101	Al Khalidiyah
3	MNH102	Al Hajj Street
3	MNH102	Al Sharaie
4	RER103	Al Hamra
5	FOF104	Al Shawqiya
6	LOL105	Batha Quraysh
7	SAR106	Al Khalidiyah
7	SAR106	Wadi Jaleel
8	ILK107	Al Auali
9	ZDT108	Al Husayniya
10	AFA109	Al Hajj Street
10	AFA109	Al Rashidiya

Query: Drivers with their shift types (LEFT JOIN)

Explanation:

Shows each driver along with all the shift types they work.
Left join ensures all drivers appear, even those with no shifts.

```
SELECT d.F_DName, d.L_DName, dst.Shift_Type
FROM Driver d
LEFT JOIN Driver_Shift_Type dst ON d.Driver_ID = dst.Driver_ID;
```

The output:

F_DName	L_DName	Shift_Type
Ahmed	Salem	Morning
Ahmed	Salem	Evening
Mazen	Ali	Morning
Fahad	Khaled	Evening
Saad	Hussein	Morning
Khaled	Majed	Morning
Amer	Omar	Evening
Sultan	Rashid	Morning
Adeab	Faisal	Morning
Talal	Amin	Evening
Salem	Saad	NULL

- **Subquery Queries**

Query: Areas assigned to more than one bus

Explanation:

Uses a subquery with HAVING COUNT > 1 to find areas covered by multiple buses.

```
SELECT *
FROM Area
WHERE Area_ID IN (
    SELECT Area_ID
    FROM Bus_Area
    GROUP BY Area_ID
    HAVING COUNT(Bus_ID) > 1
);
```

The output:

Area_ID	Area_Code	Area_Name
1	SHJ	Al Hajj Street
2	RSH	Al Rashidiya
3	HSY	Al Husayniya
4	KHD	Al Khalidiyah
NULL	NULL	NULL

Query: Drivers with more than one shift

Explanation:

Identifies drivers who have more than one assigned shift based on the shift-type table.

```
SELECT *
FROM Driver
WHERE Driver_ID IN (
    SELECT Driver_ID
    FROM Driver_Shift_Type
    GROUP BY Driver_ID
    HAVING COUNT(Shift_Type) > 1
);
```

The output:

Driver_ID	License_Num	F_DName	L_DName	Salary	Phone
1	1023456789	Ahmed	Salem	4500.00	055100001
NULL	NULL	NULL	NULL	NULL	NULL

- **VIEW Query**

Query: View showing each bus with its areas

Explanation:

Creates a view linking buses to their areas using the Bus_Area relationship table.
Useful for route management reports.

```
CREATE OR REPLACE VIEW Bus_Area_View AS
SELECT
    b.Bus_ID,
    b.Plate_Num,
    a.Area_Name
FROM Bus b
INNER JOIN Bus_Area ba ON b.Bus_ID = ba.Bus_ID
INNER JOIN Area a ON ba.Area_ID = a.Area_ID;
```

The output:

Bus_ID	Plate_Num	Area_Name
1	SAU100	Al Hajj Street
1	SAU100	Al Rashidiya
2	MAD101	Al Husayniya
2	MAD101	Al Khalidiyah
3	MNH102	Al Hajj Street
3	MNH102	Al Sharaie
4	RER103	Al Hamra
5	FOF104	Al Shawqiya
6	LOL105	Batha Quraysh
7	SAR106	Al Khalidiyah
7	SAR106	Wadi Jaleel
8	ILK107	Al Auali
9	ZDT108	Al Husayniya
10	AFA109	Al Hajj Street
10	AFA109	Al Rashidiya